

Programmation procédurale : les tableaux

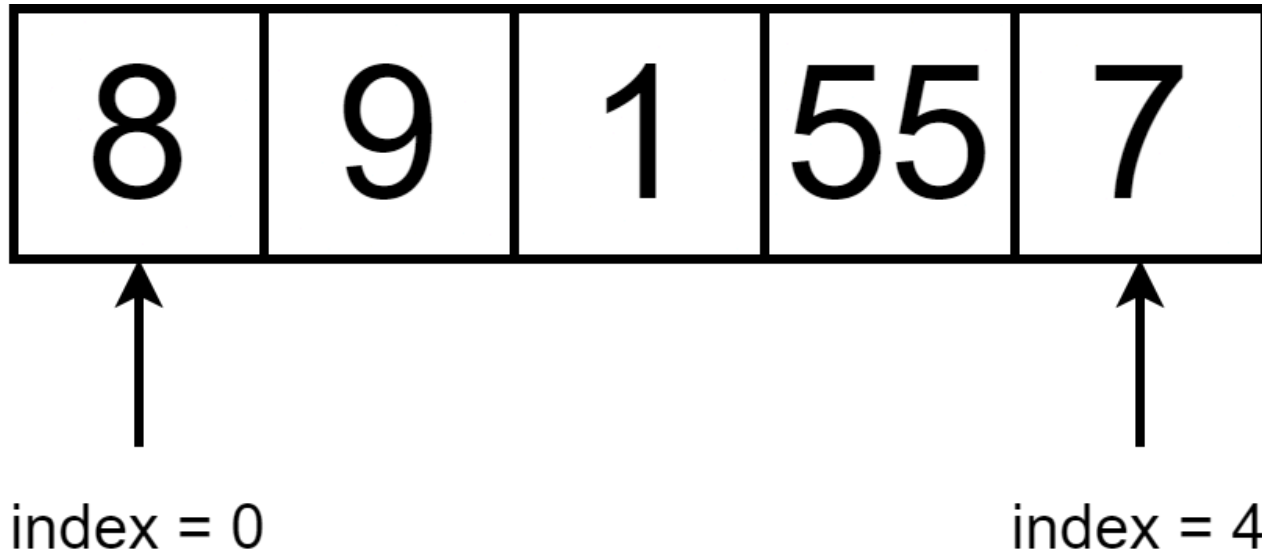


On parle de quoi ?

1. [Qu'est ce qu'un tableau ?](#)
2. [Déclaration](#)
3. [Initialisation](#)
4. [Lecture/Impression](#)
5. [Chaînes de caractères](#)
6. [Tableaux et constantes symboliques](#)

Qu'est-ce qu'un tableau ?

Un tableau est un **ensemble d'éléments successifs** en mémoire centrale accessibles via un **index**.

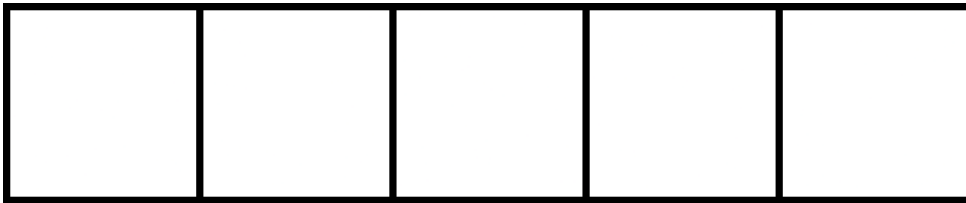


Déclaration

Un tableau est caractérisé par :

- une taille maximale (définie à l'avance)
- un nom
- un type

```
void main (void){  
  
    int monTableau[5];  
  
}
```



Déclaration

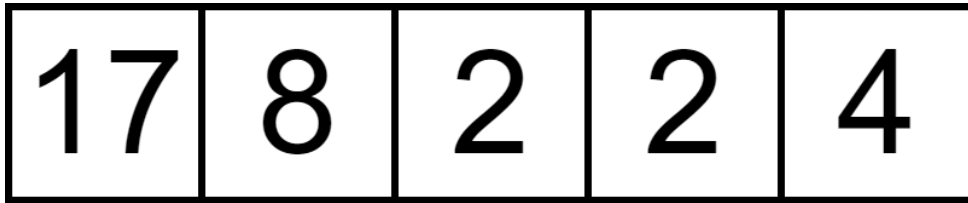
```
void main (void){  
  
    int monTableau[5];  
  
    monTableau[1] = 8;  
    monTableau[4] = 4;  
}
```

	8			4
--	---	--	--	---

Déclaration

Une fois qu'un tableau a été déclaré, **sa taille ne peut plus changer**.

```
void main(void){  
  
    int monTableau[] = {17, 8, 2, 2, 4};  
    monTableau[5] = 10; // erreur !  
  
}
```



→ **sizeof** donne la taille du tableau (le nombre d'octets réservés en mémoire)

```
void main(void){  
    int monTableau[10];  
    monTableau[5] = 10;  
  
    int nbOctets = sizeof(monTableau);  
    int taille = nbOctets / sizeof(int);  
  
}
```

Initialisation

L'initialisation d'un tableau peut se faire :

- directement avec la déclaration

```
void main(void){  
  
    int monTableau[] = {0, 0, 0, 0, 0};  
}
```

→ l'ordinateur réserve automatiquement le nombre d'octets nécessaire

- avec une répétitive

```
void main(void){  
    int monTableau[5];  
  
    for(int i = 0; i < 5; i++) monTableau[i] = 0;  
}
```

→ la taille doit être spécifiée explicitement

→ il n'existe pas de valeur par défaut !

Lecture

```
void main (void){  
    int monTableau[5];  
  
    for(int i = 0; i < 5; i++){  
        printf("Entrez le nombre %d :\n", i + 1);  
        scanf("%d", &monTableau[i]);  
    }  
}
```


Impression

```
void main (void){  
    int monTableau[5];  
  
    // le tableau se remplit...  
  
    int taille = sizeof(monTableau) / sizeof(int);  
  
    for(int i = 0; i < taille; i++){  
        printf("Nombre %d : %d\n", i + 1, monTableau[i]);  
    }  
}
```

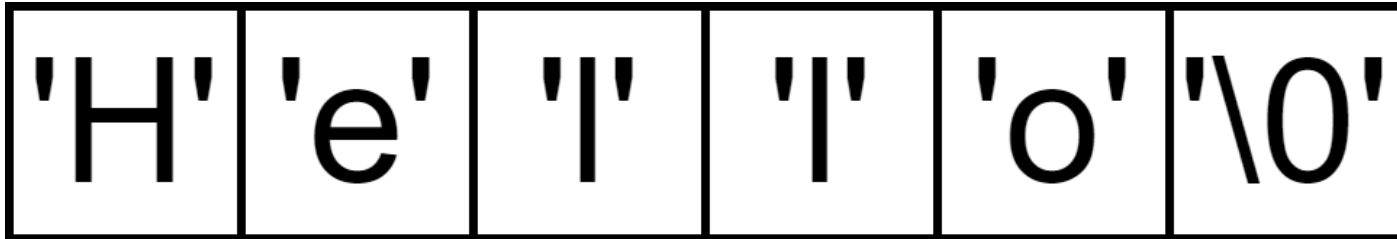
Chaînes de caractères

En C, il n'existe pas de type "chaîne de caractères" (comme le type `String` en Java par exemple).

En C, une chaîne de caractère est stockée sous la forme d'un **tableau de caractères**. Une chaîne de caractère se termine **toujours** par le caractère de fin de chaîne `\0` (code 0 de la table ASCII).

Lors d'une déclaration sans taille spécifiée, l'ordinateur réserve automatiquement le nombre d'octets nécessaires (moyennant une assignation directe).

```
// comme ça
char maChaine[] = "Hello";
// ou comme ça
char maChaine[] = {'H', 'e', 'l', 'l', 'o', '\0'};
```



Chaînes de caractères

On peut également spécifier explicitement la taille de la chaîne.

```
// comme ça
char maChaine[7] = "Hello";
// ou comme ça
char maChaine[7] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

Attention : il faut toujours prévoir un octet supplémentaire pour le stockage du `\0` !

```
char maChaine[7] = "Hello"; // ok
char maChaine[6] = "Hello"; // ko !
char maChaine[8] = "Hello"; // ok
```

Lecture

- avec `scanf`

```
scanf("%s", maChaine);
```

- avec `gets`

```
gets(maChaine);
```

Impression

- avec `printf`

```
printf("%s", maChaine);
```

- avec `puts`

```
puts(maChaine);
```

Fonctions utilitaires

Ces fonctions prédéfinies se trouvent `string.h` ou `stdlib.h`.

- `int strlen(chaine)` : donne la taille de la chaine de caractères
- `strcpy(chaineDestination , chaineSource)` : copie une chaine de caractères dans un tableau
- `strcat(chaineDestination , chaineSource)` : concatène deux chaines de caractères
- `int strcmp(chaine, autreChaine)` : compare deux chaines de caractères

Tableaux et constantes symboliques

Une **constante symbolique** permet de **remplacer**, dans le code, **des constantes par un texte significatif**.

La déclaration d'une constante symbolique se place toujours :

- au début du code source (fichier avec l'extension **.c**) ou
- dans un fichier d'en-tête ("header file", un fichier avec l'extension **.h**).

Le syntaxe est :

```
#define MA_CONSTANTE valeur
```

Exemple :

```
#define TAILLE_NOM 100
#define NB_COURS 10

void main (void){
    char nom[TAILLE_NOM];
    char deuxiemeNom[TAILLE_NOM];
    int codeCours[NB_COURS];
}
```

Un exemple final

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#define TAILLE_MDP 50

void main(void) {
    char mdp[TAILLE_MDP] = "INDBG2022";
    char mdpUtilisateur[TAILLE_MDP];
    int longueur = strlen(mdp);
    int taille = sizeof(mdp); // sizeof(char) = 1

    printf("Longueur : %d\nTaille: %d\n", longueur, taille);

    printf("Entrez votre mot de passe :\n");
    gets(mdpUtilisateur);
    if (strcmp(mdp, mdpUtilisateur) == 0) puts("Mot de passe correct !");
    else puts("Erreur : mot de passe incorrect");
    strcpy(mdpUtilisateur, mdp);
    puts(mdpUtilisateur);
    strcat(mdp, "2014");
    puts(mdp);
}
```